

C++ Assignment 1: Encapsulation and Abstraction

Create a class BankAccount with private data members and public methods to initialize, deposit, withdraw (validate minimum balance), and display details.

Code:

```
#include <iostream>
using namespace std;

class BankAccount {
private:
    int accountNumber;
    string name;
    float balance;

public:
    void initialize(int accNo, string accName, float bal) {
        accountNumber = accNo;
        name = accName;
        balance = bal;
    }

    void deposit(float amount) {
        balance += amount;
        cout << "Amount Deposited: " << amount << endl;
    }

    void withdraw(float amount) {
        if (balance - amount >= 500) // maintain min balance
            balance -= amount;
        else
            cout << "Insufficient balance or minimum balance rule violated!" << endl;
    }

    void display() {
        cout << "\nAccount Details:\n";
        cout << "Account Number: " << accountNumber << endl;
        cout << "Name: " << name << endl;
        cout << "Balance: " << balance << endl;
    }
};

int main() {
    BankAccount b;
    b.initialize(1001, "Soham", 5000);
    b.deposit(1500);
    b.withdraw(2000);
    b.display();
    return 0;
}
```

Expected Output:

Amount Deposited: 1500

Account Details:
Account Number: 1001
Name: Soham
Balance: 4500

C++ Assignment 2: Single and Multilevel Inheritance

Design classes for a Student Result System: Person -> Student -> Exam. Compute total and average marks, and display results.

Code:

```
#include <iostream>
using namespace std;

class Person {
public:
    string name;
    int age;
};

class Student : public Person {
public:
    int rollNo;
    string course;
};

class Exam : public Student {
public:
    float marks1, marks2, marks3;
    void input() {
        cout << "Enter Name Age RollNo Course Marks1 Marks2 Marks3: ";
        cin >> name >> age >> rollNo >> course >> marks1 >> marks2 >> marks3;
    }
    void display() {
        float total = marks1 + marks2 + marks3;
        float avg = total / 3;
        cout << "\nName: " << name << "\nAge: " << age << "\nRoll No: " << rollNo;
        cout << "\nCourse: " << course << "\nTotal: " << total << "\nAverage: " << avg << endl;
    }
};

int main() {
    Exam e;
    e.input();
    e.display();
    return 0;
}
```

Expected Output:

Sample Input: Soham 20 1 CS 90 85 80

Output:

Name: Soham

Age: 20

Roll No: 1

Course: CS

Total: 255

Average: 85

C++ Assignment 3: Hierarchical and Multiple Inheritance

Create an Employee Payroll System with Employee (empId, name), Department (deptName, location), and Salary (basic, hra, da). Calculate total salary.

Code:

```
#include <iostream>
using namespace std;

class Employee {
public:
    int empId;
    string name;
};

class Department {
public:
    string deptName;
    string location;
};

class Salary : public Employee, public Department {
public:
    float basic, hra, da;
    void input() {
        cout << "Enter EmpId Name DeptName Location Basic HRA DA: ";
        cin >> empId >> name >> deptName >> location >> basic >> hra >> da;
    }
    void display() {
        float total = basic + hra + da;
        cout << "\nEmployee ID: " << empId << "\nName: " << name
              << "\nDepartment: " << deptName << "\nLocation: " << location
              << "\nTotal Salary: " << total << endl;
    }
};

int main() {
    Salary s;
    s.input();
    s.display();
    return 0;
}
```

Expected Output:

Sample Input: 1 Soham IT Pune 20000 5000 3000

Output:

Employee ID: 1

Name: Soham

Department: IT

Location: Pune

Total Salary: 28000

C++ Assignment 4: Polymorphism (Compile-time and Runtime)

1) Function overloading for area() for circle, rectangle, triangle. 2) Base class Shape with virtual displayArea(), derived Circle and Rectangle implement it.

Code:

```
#include <iostream>
using namespace std;

// Compile-time Polymorphism (Function Overloading)
float area(float r) { return 3.14f * r * r; }
float area(float l, float b) { return l * b; }
float area(float b, int h) { return 0.5f * b * h; }

// Runtime Polymorphism (Virtual Functions)
class Shape {
public:
    virtual void displayArea() { cout << "Generic Shape" << endl; }
};

class Circle : public Shape {
public:
    void displayArea() override { cout << "Area of Circle = " << area(5) << endl; }
};

class Rectangle : public Shape {
public:
    void displayArea() override { cout << "Area of Rectangle = " << area(4, 5) << endl; }
};

int main() {
    cout << "Area of Circle (Function Overload): " << area(3) << endl;
    cout << "Area of Rectangle: " << area(4, 5) << endl;
    cout << "Area of Triangle: " << area(6, 4) << endl;

    Shape* s;
    Circle c;
    Rectangle r;
    s = &c;
    s->displayArea();
    s = &r;
    s->displayArea();
    return 0;
}
```

Expected Output:

```
Output:
Area of Circle (Function Overload): 28.26
Area of Rectangle: 20
Area of Triangle: 12
Area of Circle = 78.5
Area of Rectangle = 20
```

C++ Assignment 5: Constructors and Destructors

Create class Student with default, parameterized, and copy constructors, and a destructor to show message when destroyed.

Code:

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int rollNo;
    float marks;

    Student() { cout << "Default Constructor Called" << endl; }
    Student(string n, int r, float m) {
        name = n; rollNo = r; marks = m;
        cout << "Parameterized Constructor Called" << endl;
    }
    Student(const Student &s) {
        name = s.name; rollNo = s.rollNo; marks = s.marks;
        cout << "Copy Constructor Called" << endl;
    }
    ~Student() { cout << "Destructor Called for " << name << endl; }

    void display() {
        cout << "Name: " << name << " | Roll No: " << rollNo << " | Marks: " << marks << endl;
    }
};

int main() {
    Student s1("Soham", 1, 89.5);
    s1.display();
    Student s2 = s1;
    s2.display();
    return 0;
}
```

Expected Output:

Expected Output (order of destructor messages at program end may vary):
Parameterized Constructor Called
Name: Soham | Roll No: 1 | Marks: 89.5
Copy Constructor Called
Name: Soham | Roll No: 1 | Marks: 89.5
Destructor Called for Soham
Destructor Called for Soham

C++ Assignment 6: File Handling

Maintain a student record file student.txt: store rollNo, name, marks; read and display all records; append new data.

Code:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream fout("student.txt", ios::app);
    int rollNo;
    string name;
    float marks;

    cout << "Enter Roll No, Name, Marks: ";
    cin >> rollNo >> name >> marks;
    fout << rollNo << " " << name << " " << marks << endl;
    fout.close();

    ifstream fin("student.txt");
    cout << "\nStudent Records:\n";
    while (fin >> rollNo >> name >> marks) {
        cout << rollNo << " " << name << " " << marks << endl;
    }
    fin.close();
    return 0;
}
```

Expected Output:

Sample Input: 1 Soham 90

Student Records:

1 Soham 90

C++ Assignment 7: Exception Handling

Perform division of two numbers and handle division by zero and invalid input.

Code:

```
#include <iostream>
#include <stdexcept>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two numbers: ";
    if (!(cin >> a >> b)) {
        cout << "Invalid input!" << endl;
        return 0;
    }

    try {
        if (b == 0)
            throw runtime_error("Division by Zero!");
        cout << "Result: " << (float)a / b << endl;
    } catch (runtime_error &e) {
        cout << "Error: " << e.what() << endl;
    }

    cout << "Program ended normally." << endl;
    return 0;
}
```

Expected Output:

Example 1 Input: 10 0

Output:

Error: Division by Zero!

Program ended normally.

Example 2 Input: 10 2

Output:

Result: 5

Program ended normally.

Java Assignment 1: Encapsulation and Abstraction

Student class with private data members and getter/setter methods. Display student details.

Code:

```
class Student {
    private String name;
    private int rollNo;
    private float marks;

    // Setter methods
    void setName(String n) { name = n; }
    void setRollNo(int r) { rollNo = r; }
    void setMarks(float m) { marks = m; }

    // Display method
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Marks: " + marks);
    }

    public static void main(String[] args) {
        Student s = new Student();
        s.setName("Soham");
        s.setRollNo(1);
        s.setMarks(89.5f);
        s.display();
    }
}
```

Expected Output:

```
Name: Soham
Roll No: 1
Marks: 89.5
```


Java Assignment 2: Inheritance (Single & Multilevel)

Create class hierarchy Person -> Employee -> Manager and display manager details using constructors and super keyword.

Code:

```
class Person {
    String name;
    int age;
}

class Employee extends Person {
    int empId;
    double salary;
}

class Manager extends Employee {
    String department;

    Manager(String n, int a, int id, double s, String d) {
        name = n; age = a; empId = id; salary = s; department = d;
    }

    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Emp ID: " + empId);
        System.out.println("Salary: " + salary);
        System.out.println("Department: " + department);
    }

    public static void main(String[] args) {
        Manager m = new Manager("Soham", 21, 101, 55000, "IT");
        m.display();
    }
}
```

Expected Output:

```
Output:
Name: Soham
Age: 21
Emp ID: 101
Salary: 55000.0
Department: IT
```

Java Assignment 3: Polymorphism (Compile-time & Runtime)

Method overloading for area() and method overriding for draw() in derived classes.

Code:

```
class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

class OverloadDemo {
    double area(double r) { return 3.14 * r * r; }
    double area(double l, double b) { return l * b; }
    double area(double b, int h) { return 0.5 * b * h; }

    public static void main(String[] args) {
        OverloadDemo o = new OverloadDemo();
        System.out.println("Area of Circle: " + o.area(3));
        System.out.println("Area of Rectangle: " + o.area(4, 5));
        System.out.println("Area of Triangle: " + o.area(6, 4));

        Shape s;
        s = new Circle();
        s.draw();
        s = new Rectangle();
        s.draw();
    }
}
```

Expected Output:

```
Output:
Area of Circle: 28.26
Area of Rectangle: 20.0
Area of Triangle: 12.0
Drawing a Circle
Drawing a Rectangle
```

Java Assignment 4: Threads – Part A (Thread Creation & Lifecycle)

Create threads by extending Thread and by implementing Runnable. Use start(), sleep(), join(), interrupt().

Code:

```
class MyThread1 extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Thread 1: " + i);
            try { Thread.sleep(300); } catch (Exception e) {}
        }
    }
}

class MyThread2 implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Thread 2: " + i);
            try { Thread.sleep(300); } catch (Exception e) {}
        }
    }
}

public class ThreadDemo {
    public static void main(String[] args) throws InterruptedException {
        MyThread1 t1 = new MyThread1();
        Thread t2 = new Thread(new MyThread2());

        t1.start();
        t1.join(); // wait until t1 finishes
        t2.start();
    }
}
```

Expected Output:

Sample Output:

```
Thread 1: 1
Thread 1: 2
Thread 1: 3
Thread 2: 1
Thread 2: 2
Thread 2: 3
```

Java Assignment 5: Threads – Part B (Synchronization & Thread Safety)

Demonstrate synchronization to avoid race conditions with a shared counter, and show result with synchronized keyword.

Code:

```
class Counter {
    private int count = 0;

    synchronized void increment() {
        count++;
    }

    int getCount() {
        return count;
    }
}

public class SyncDemo {
    public static void main(String[] args) throws Exception {
        Counter c = new Counter();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) c.increment();
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) c.increment();
        });

        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("Final Count (with sync): " + c.getCount());
    }
}
```

Expected Output:

Output:

Final Count (with sync): 2000

Java Assignment 6: Threads – Part C (Inter-Thread Communication)

Implement Producer-Consumer using a shared buffer and wait/notify/notifyAll methods.

Code:

```
class SharedBuffer {
    private int data;
    private boolean available = false;

    synchronized void produce(int value) throws InterruptedException {
        while (!available) wait();
        data = value;
        System.out.println("Produced: " + data);
        available = true;
        notify();
    }

    synchronized int consume() throws InterruptedException {
        while (!available) wait();
        System.out.println("Consumed: " + data);
        available = false;
        notify();
        return data;
    }
}

public class ProducerConsumer {
    public static void main(String[] args) {
        SharedBuffer buffer = new SharedBuffer();

        Thread producer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                try { buffer.produce(i); } catch (Exception e) {}
            }
        });

        Thread consumer = new Thread(() -> {
            for (int i = 1; i <= 5; i++) {
                try { buffer.consume(); } catch (Exception e) {}
            }
        });

        producer.start();
        consumer.start();
    }
}
```

Expected Output:

Sample Output (order may vary due to scheduling):

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
```

Java Assignment 7: Exception Handling

Handle `ArithmeticException` (division by zero) and `InputMismatchException` for invalid input.

Code:

```
import java.util.*;

public class ExceptionDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter two numbers: ");
            int a = sc.nextInt();
            int b = sc.nextInt();
            int result = a / b;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero!");
        } catch (InputMismatchException e) {
            System.out.println("Error: Invalid input type!");
        } finally {
            System.out.println("Program ended.");
        }
    }
}
```

Expected Output:

Output Example 1:

Enter two numbers: 10 0

Error: Division by zero!

Program ended.

Output Example 2:

Enter two numbers: 10 ten

Error: Invalid input type!

Program ended.

Java Assignment 8: Collection Framework

Use ArrayList to store student details, HashSet to remove duplicates, HashMap to map roll numbers to names, sort using Collections.sort(), and iterate.

Code:

```
import java.util.*;

public class CollectionDemo {
    public static void main(String[] args) {
        // ArrayList
        ArrayList<String> students = new ArrayList<>();
        students.add("Soham");
        students.add("Riya");
        students.add("Riya"); // duplicate

        System.out.println("ArrayList: " + students);

        // HashSet
        HashSet<String> uniqueNames = new HashSet<>(students);
        System.out.println("HashSet (no duplicates): " + uniqueNames);

        // HashMap
        HashMap<Integer, String> map = new HashMap<>();
        map.put(1, "Soham");
        map.put(2, "Riya");

        System.out.println("HashMap: " + map);

        // Sorting
        Collections.sort(students);
        System.out.println("Sorted List: " + students);

        // Iterator (enhanced for-loop)
        System.out.println("Iterating list:");
        for (String s : students) System.out.println(s);
    }
}
```

Expected Output:

```
Output:
ArrayList: [Soham, Riya, Riya]
HashSet (no duplicates): [Riya, Soham]
HashMap: {1=Soham, 2=Riya}
Sorted List: [Riya, Riya, Soham]
Iterating list:
Riya
Riya
Soham
```

Java Assignment 9: File Handling (Simple Program)

Create data.txt, accept and write user input (name and marks), then read and display file contents.

Code:

```
import java.io.*;
import java.util.*;

public class FileIODemo {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        FileWriter fw = new FileWriter("data.txt", true);
        BufferedWriter bw = new BufferedWriter(fw);

        System.out.print("Enter name and marks: ");
        String name = sc.next();
        int marks = sc.nextInt();

        bw.write(name + " " + marks);
        bw.newLine();
        bw.close();

        System.out.println("\nReading from file:");
        BufferedReader br = new BufferedReader(new FileReader("data.txt"));
        String line;
        while ((line = br.readLine()) != null)
            System.out.println(line);
        br.close();
    }
}
```

Expected Output:

Sample Interaction:

Enter name and marks: Soham 90

Reading from file:

Soham 90